

Tehničko veleučilište u Zagrebu
Informatičko-računarski odjel

Duboko učenje

Tehnike treniranja neuronskih mreža



Neke od zanimljivih tehnika

Treniranje na više GPU-ova

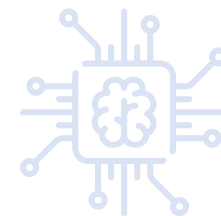
Korištenje rasporeda za promjenu stope učenja

Transferirano učenje (engl. Transfer learning)

N-shot learning

Reinforcement learning

Automatsko podešavanje hiper parametara



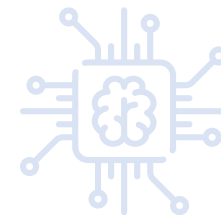
Treniranje na više GPU-ova

Vrlo često je problem da se treniranje na jednoj grafičkoj kartici ne odvija dovoljno brzo

Iz istog razloga, moguće je koristiti više grafičkih kartica.

S obzirom na arhitekturu mreže, pojedine slojeve mreže moguće je čak držati na različitim grafičkim karticama

Za treniranje i klasično pokretanje neuronske mreže na više GPU-ova u PyTorchu se koriste `DataParallel` ili `DistributedDataParallel` klasa



DataParallel vs. DistributedDataParallel

DataParallel je jedan proces s više dretvi i radi na jednom računalu

DistributedDataParallel je višeprocen i radi na jednom ili više računala

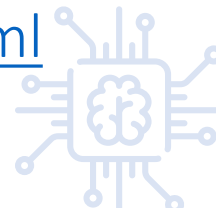
DataParallel je nešto sporiji, a DistributedDataParallel je nešto brži

DistributedDataParallel zna raditi s modelima koji se prostiru preko više GPU-ova, dok DataParallel ne zna

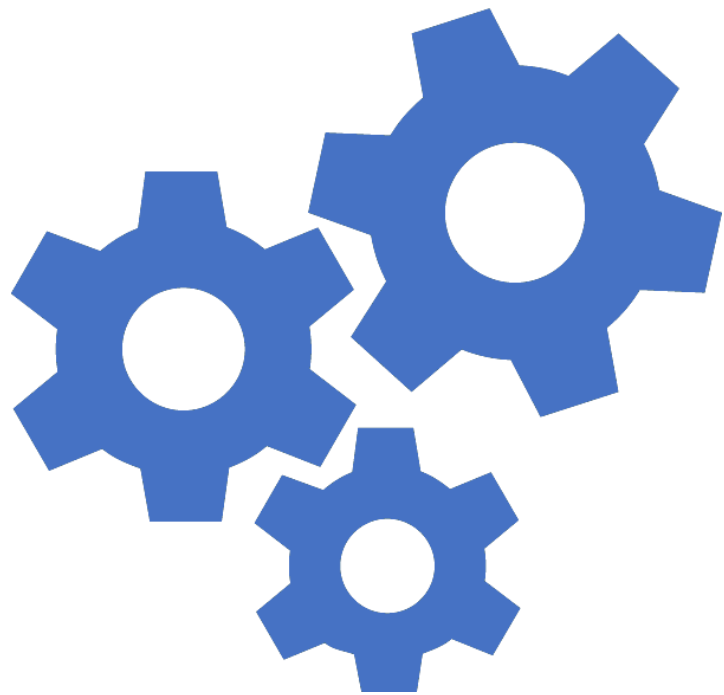
Naravno da postoji kvaka – DataParallel je jednostavniji za korištenje i debugging

DistributedDataParallel poprilično kompleksan za korištenje i implementaciju te se u pravilu koristi na oblaku.

Više na: <https://www.telesens.co/2019/04/04/distributed-data-parallel-training-using-pytorch-on-aws/> i https://pytorch.org/tutorials/intermediate/ddp_tutorial.html



DataParallel



Korištenje DataParallel-a



Raspored stope učenja

Stopu učenja moguće je u PyTorch-u jednostavno podesiti pomoću `optim.lr_scheduler` komponente

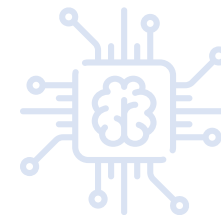
Sadrži velik broj rasporeda (engl. scheduler) kojima se podešava stopa učenja

Ovisno o potrebama problema – postoje oni koji smanjuju za deltu, za neki broj, za koeficijent, a smanjivanje se događa ovisno o koraku ili nekom drugom uvjetu

Stopu je potrebno često smanjivati kako ne bi „preskočili” globalni minimum

Za detaljan pregled mogućih funkcionalnosti rasporeda stope učenja posjetiti:

<https://pytorch.org/docs/stable/optim.html>



Transferirano učenje

Transferirano učenje nije novi koncept, već datira iz 1995, ali se počeo upotrebljavati od 2010-ih

To je tehnika učenja u kojoj naučeno znanje neke neuronske mreže prenesemo na drugi problem

Prva motivacija je ta da nije potrebno dugotrajno treniranje cijele neuronske mreže, već je dovoljno „dotrenirati” neki od postojećih modela

Druga motivacija je da nam nije potrebno toliko podataka za „dotreniravanje”

Tradicionalno se radi treniranje neuronske mreže na svakom setu podataka posebno

Transferirano učenje omogućava treniranje na jednom setu podataka i „dotreniravanje” na drugom setu podataka



Transferirano učenje

Što transferirati?

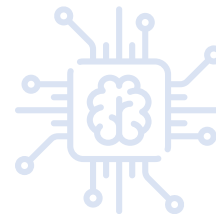
Treba razmisliti koji dio znanja je moguće prenijeti. Potrebno je vidjeti koji dio znanja je specifičan za inicijalnu neuronsku mrežu.

Kada transferirati?

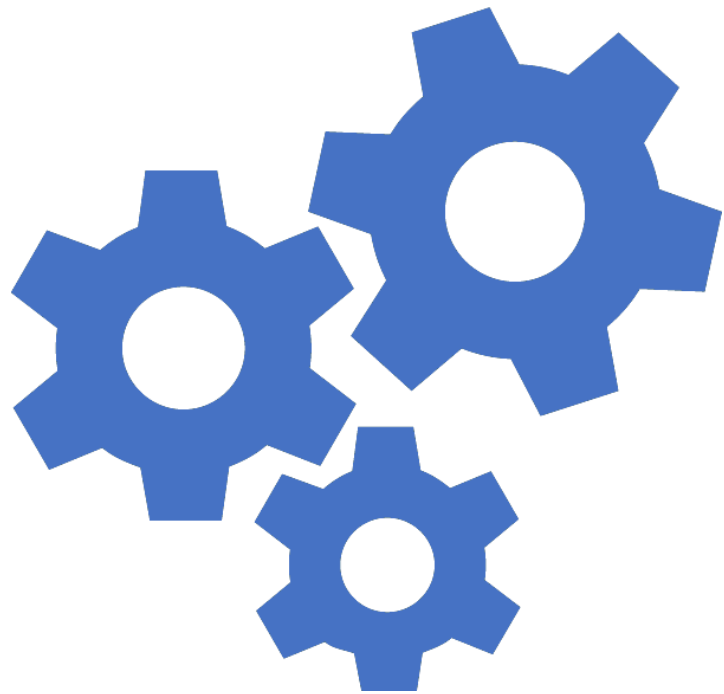
Postoje slučajevi u kojima će transferirano učenje dati lošije rezultate. Konkretno, domena problema je dovoljno drugačija. Potrebno je biti oprezan oko tog problema.

Kako transferirati?

Generalno se mijenjaju težine posljednjih slojeva u konvolucijskim mrežama, ali postoje različite tehnike.



Transferirano učenje



Primjer transferiranog učenja na unaprijed treniranoj ResNet mreži.

Primjer transferiranog učenja s promjenom rasporeda.



N-shot learning

To su tehnike učenja na malom broju primjeraka

Postoje:

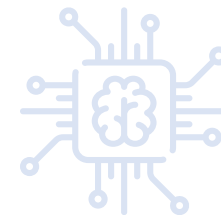
- Zero-shot learning

- One-shot learning

- Few-shot learning

Zapravo su ekstremna podskupina transferiranog učenja

Više: <https://blog.floydhub.com/n-shot-learning/>



Zero-shot learning

Tradicionalno učenje se temelji na velikom broju podataka

Podaci nisu određeni klasom

Mreža mapira neoznačene podatke

Koriste se semantički podaci koji se mapiraju s vizualnim karakteristikama slike

Potrebni su pomoćni podaci u obliku atributa

Razmišljajmo na način:

Je li moguće klasificirati objekt bez da ga se vidi?

Je, moguće je ako nam ga netko dobro opiše.

Primjer:

<https://medium.com/@harenb/zero-shot-learning-using-embeddings-a-python-example-cffacb94a034>



One-shot learning

Mreža vidi jedan primjer svake klase

Najčešće se koriste sijamske neuronske mreže

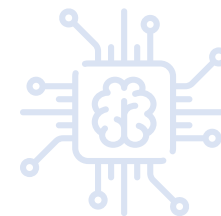
Dvije ili više identičnih podmreža koje imaju istu konfiguraciju i iste parametre i težine

Parametri se osvježavaju preko obje podmreže

Koriste se kako bi se pronašle značajke (engl. feature) ulaza uspoređujući vektore značajki (engl. feature vectors)

Primjer mreže: <https://github.com/fangpin/siamese-pytorch>

Najbitniji dio je dataset i u forwardu mreže



Few-shot learning

To je fleksibilnija varijanta one-shot learninga

Postoji više od jednog primjera (najčešće 2-5)

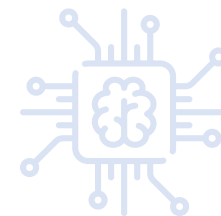
Može se koristiti sijamska neuronska mreža

Mreža vidi jedan primjer svake klase

Koriste se prototipične mreže

<https://github.com/orobix/Prototypical-Networks-for-Few-shot-Learning-PyTorch>

Također se mogu koristiti sijamske neuronske mreže



Reinforcement learning

To je treniranje na način da računalo pomoću modela radi sekvence odluka

Mora ostvariti cilj u kompleksnom i ne unaprijed određenom okruženju

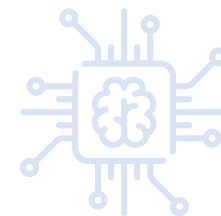
Računalo koristi metodu pokušaja pogreške

Koristi se sustav nagrađivanja i kažnjavanja

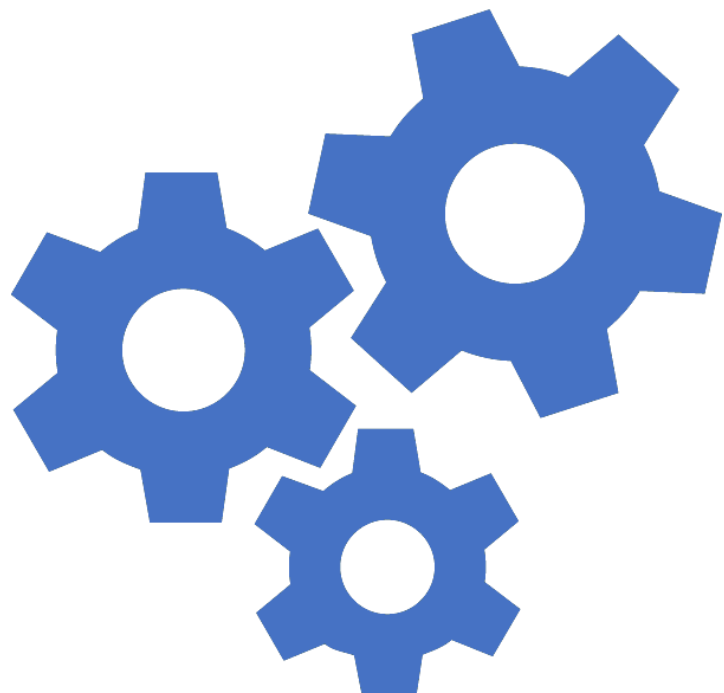
Model nije svjestan svoje okoline – na njemu je da riješi problem

Najčešće se koristi za autonomna vozila

Mi ćemo ga koristiti za prelazak igre. :)

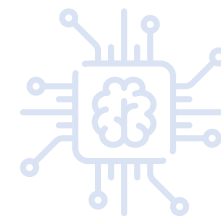


Reinforcement learning



Držanje ravnoteže

Prolaženje svima najdraže igre flappy birds



Automatsko podešavanje hiperparametara

Postoji nekoliko metoda kojima se mogu automatski prilagođavati hiper-parametri

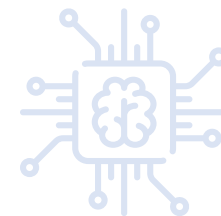
Najčešće se koriste:

- Rešetkasto pretraživanje (engl. Grid search)

- Nasumično pretraživanje (engl. Random search)

- Bayesova optimizacija (engl. Bayesian optimization)

- Hiperband optimizacija (engl. Hyperband optimization)



Rešetkasto pretraživanje

U rešetkastom pretraživanju odabiremo skup vrijednosti za svaki parametar, a skup pokusa formira se sastavljanjem svake moguće kombinacije vrijednosti

Jednostavno je provesti i paralelizirati

Problem je dimenzionalnosti jer broj zajedničkih vrijednosti eksponencijalno raste s brojem hiper-parametara

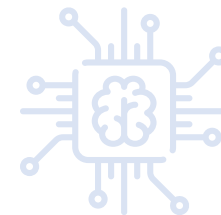


Nasumično pretraživanje

Nasumično pretraživanje smatra se varijacijom rešetkastog pretraživanja, gdje se neovisno odabiru slučajne vrijednosti iz jednolike gustoće iz istog konfiguracijskog prostora kao što bi bilo obuhvaćeno rešetkastim pretraživanjem

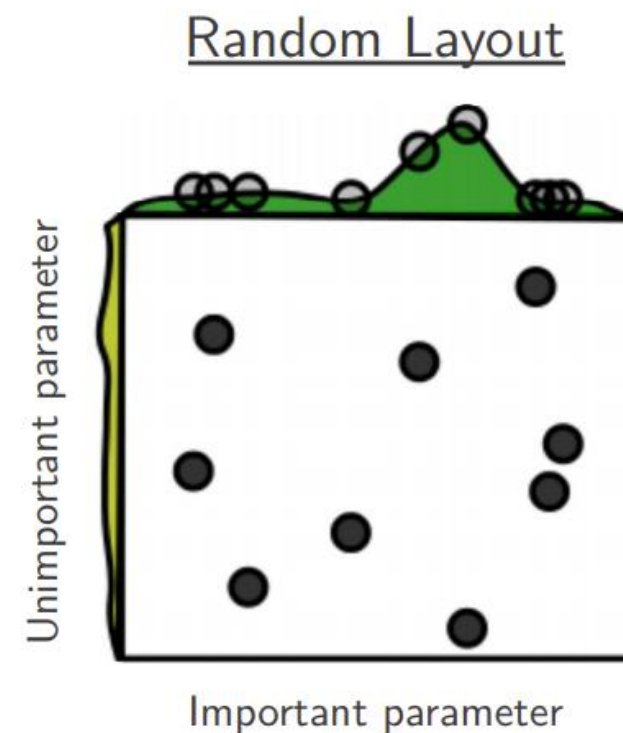
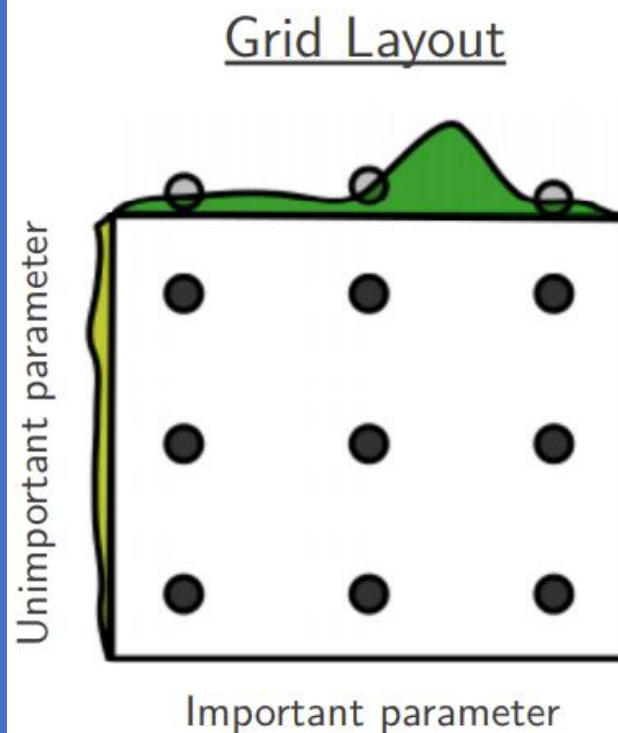
Nasumično pretraživanje ima sve praktične prednosti kao i rešetkasto pretraživanje, ali je učinkovitije u prostorima s više dimenzija, dok je nešto lošije u prostoru s manjim brojem dimenzija

Učinkovitije je od pretraživanja mreže u visokodimenzionalnim prostorima jer je hiperparametarska funkcija koju želimo optimizirati osjetljivija na promjene nekih dimenzija od drugih



Usporedba rešetkastog i nasumičnog pretraživanja

Rešetkasto i nasumično
pretraživanje u
dvodimenzionalnom
prostoru



Preuzeto iz rada:

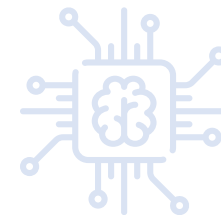
<https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>

Bayesova optimizacija

Bayesova optimizacija traži vrijednost koja minimizira ciljnu funkciju na način da kreira surogatnu funkciju na temelju evaluacije prošlih rezultata ciljne funkcije

Surogat je jeftinije optimizirati od cilja, pa se sljedeće ulazne vrijednosti za procjenu odabiru primjenom kriterija na surogat (očekivano poboljšanje)

Bayesove metode razlikuju se od slučajnog pretraživanja ili pretraživanja mreže po tome što koriste prošle rezultate vrednovanja za odabir sljedećih vrijednosti za procjenu



Bayesova optimizacija

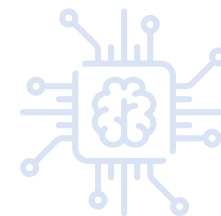
U slučaju optimizacije hiperparametara, ciljna funkcija je pogreška provjere valjanosti modela neuronske mreže pomoću skupa hiperparametara

Cilj je pronaći hiperparametre koji daju najmanju pogrešku na skupu provjere valjanosti u nadi da će se ovi rezultati generalizirati na skup ispitivanja

Procjena ciljne funkcije skupa je jer zahtijeva treniranje modela neuronske mreže s određenim skupom hiperparametara

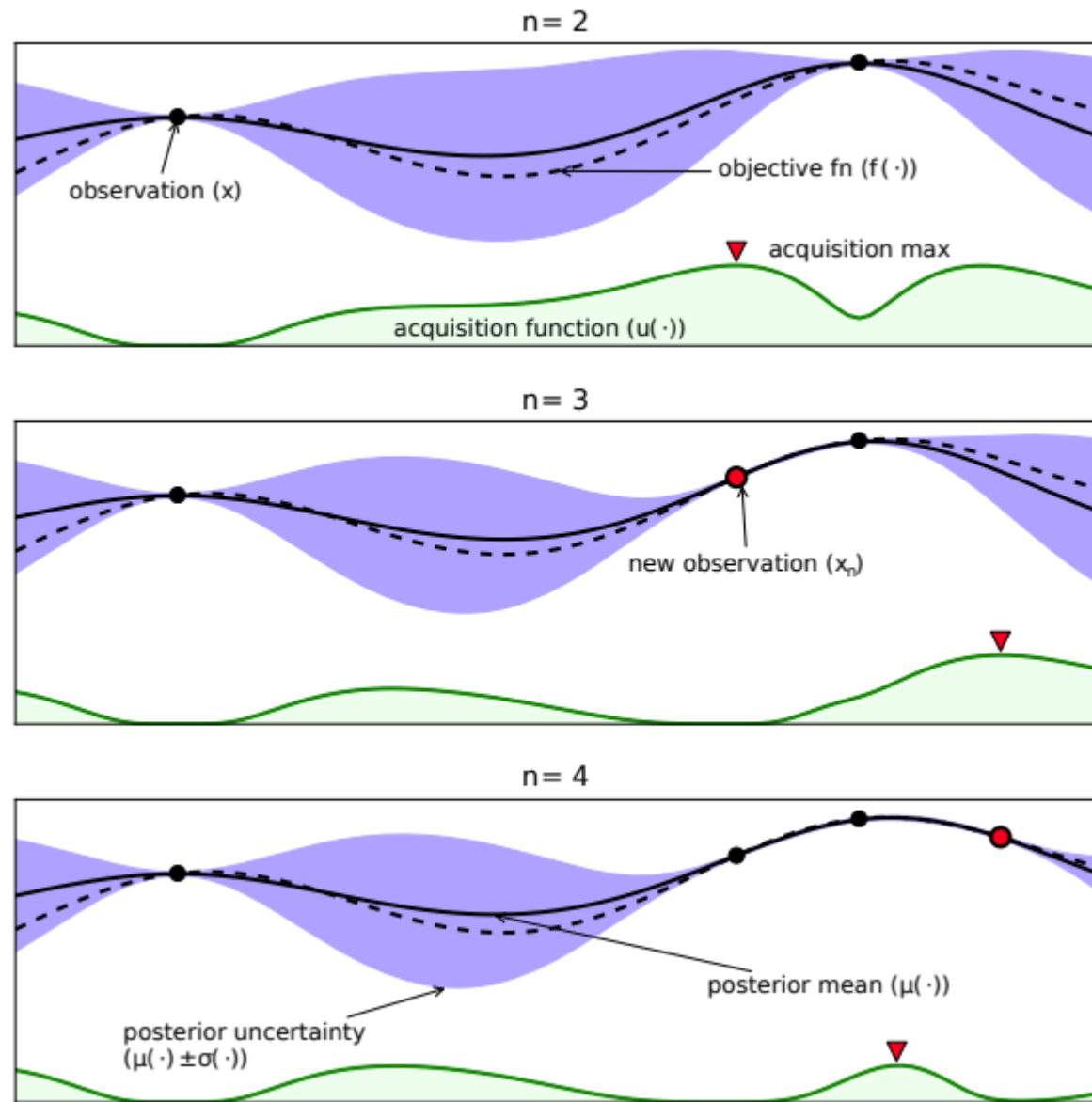
U idealnom slučaju, želimo metodu koja može istražiti prostor pretraživanja, istovremeno ograničavajući procjene loših izbora hiperparametara

Bayesovo podešavanje hiperparametara koristi se kontinuirano ažuriranim modelom vjerojatnosti kako bi se "koncentrirao" na perspektivne hiperparametre rezonirajući iz prošlih rezultata



Bayesova optimizacija

Tri iteracije Bayesove optimizacije



Preuzeto iz rada:

<https://www.cs.ox.ac.uk/people/nando.defreitas/publications/BayesOptLoop.pdf>

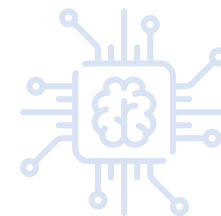
Hiperband optimizacija

Bayesova optimizacija je učinkovitija i brža od slučajnog pretraživanja no ima problem kod istodobnog uklapanja i optimizacije visokodimenzionalne, nekonveksne funkcije s nepoznatom glatkoćom i evaluacija s velikim šumom (nema glatku krivulju)

Hiperband je tehnika koja izbjegava ovaj problem čineći minimalne pretpostavke i usredotočena je na rješavanje načina raspodjele resursa među slučajno uzorkovanim konfiguracijama hiperparametara

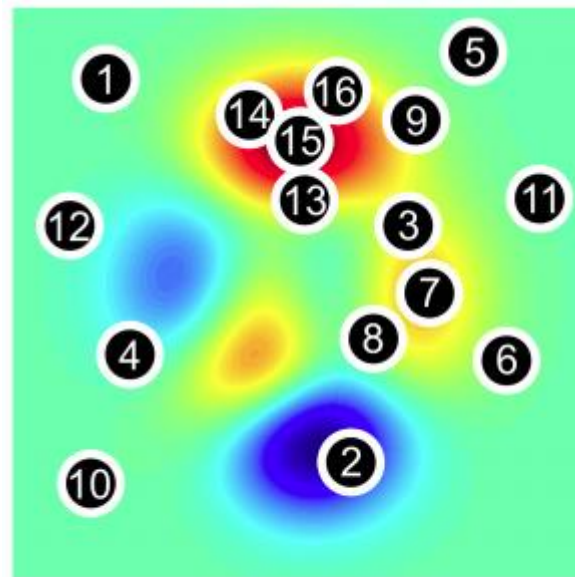
Hiperband je u osnovi varijacija nasumičnog pretraživanja koja koristi strategiju ranog zaustavljanja

Hyperband evaluira više konfiguracija hiperparametara i konvergira brže od Bayesove optimizacije

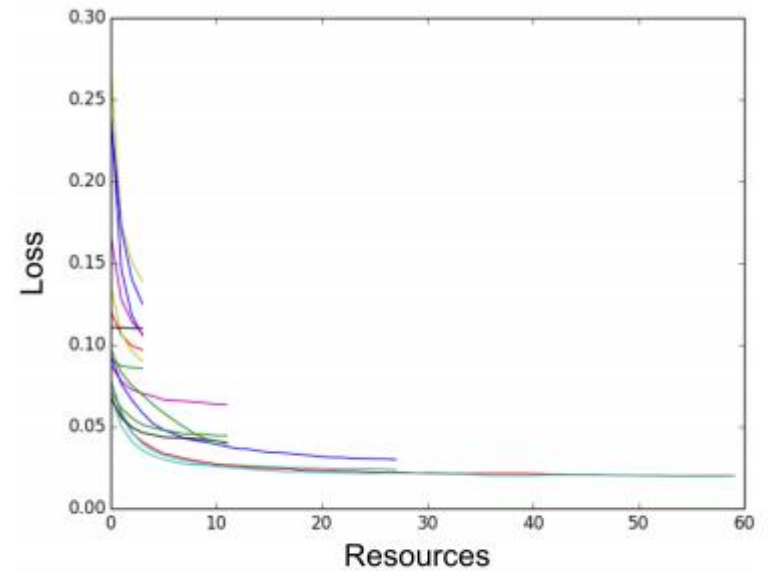


Hiperband optimizacija

- a) Heatmapa pokazuje pogrešku validacije (crveno su niže greške)
- b) Graf pokazuje pogrešku validacije za svaku konfiguraciju



(a) Configuration Selection



(b) Configuration Evaluation

Preuzeto iz rada:

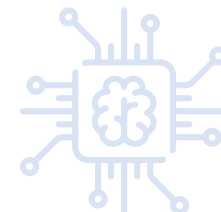
<https://arxiv.org/pdf/1603.06560.pdf>

Podrška za PyTorch

PyTorch nema podršku „out-of-the-box“, već je potrebno koristiti neke od dodatnih biblioteka kao npr:

<https://optuna.org/>

Slobodno se poigrajte s Optunom nakon što mislite da ste podesili sve hiperparametre. ;)



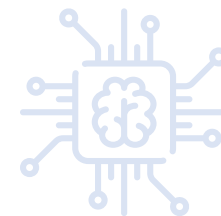
Dodatna literatura

Pregled raznih tehnika treniranja i traženja arhitekture neuronske mreže moguće je pronaći u sljedećim izvorima:

<https://arxiv.org/pdf/2005.11074.pdf>

<https://lilianweng.github.io/lil-log/2020/08/06/neural-architecture-search.html>

<https://www.automl.org/automl/literature-on-neural-architecture-search/>



Pitanja?